A Project Report on

FAKE JOB POSTING DETECTION

USING MACHINE LEARNING

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS (BCA)

Submitted by

Anjali Pandey

Roll Number: 240004

Under the Guidance of

Ms. Priyanka Pal, Data Science Trainer

Starex University

Department of Computer Science Engineering

Gurgaon, Haryana

2024-2027

DECLARATION

I, Anjali Pandey, Roll Number 240004, hereby declare that the project report entitled “Fake Job Posting Detection Using Machine Learning”, submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Computer Applications (BCA), is an original piece of work carried out by me under the guidance of Ms. Priyanka Pal.

I further declare that this report, or any part of it, has not been submitted previously by me or by any other person for the award of any degree, diploma, or similar title of any other university or institution. All sources of information, data, and literature used in the preparation of this report have been duly acknowledged in the References section.

Date: 25 July 2026

Place: Starex University

Anjali Pandey
Roll Number: 240004

CERTIFICATE

This is to certify that the project report entitled “Fake Job Posting Detection Using Machine Learning” is a bona fide work carried out by Anjali Pandey (Roll Number: 240004), submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Computer Applications (BCA) from Starex University, during the academic year 2024-2027.

The work embodied in this report has been carried out under my supervision and guidance. This report has not been submitted elsewhere for the award of any other degree or diploma, in full or in part, to the best of my knowledge.

Date: 25 July 2026

Place: Starex University

_____ _____

Project Guide

Ms. Priyanka Pal

Head of Department

Ms. Ritu Malik Siwatch

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, Ms. Priyanka Pal, for the valuable guidance, continuous encouragement, and constructive feedback provided throughout the course of this project. Their insights into machine learning practices and academic writing greatly shaped the direction of this work.

I am also thankful to the Head of the Department of Computer Science Engineering, Ms. Ritu Malik Siwatch, and to all the faculty members who provided the technical foundation necessary to undertake a project of this nature.

I extend my appreciation to Starex University for providing the infrastructure, laboratory facilities, and learning resources that made this project possible.

Finally, I am grateful to my family and friends for their unwavering support and motivation during the course of this project, and to the open-source community whose datasets, libraries, and documentation formed the backbone of this work.

Anjali Pandey
Roll Number: 240004

TABLE OF CONTENTS

Chapter 1: Introduction

1.1 Background

The growth of the internet has fundamentally changed the recruitment landscape. Job seekers today rely heavily on online job portals, professional networking sites, and social media platforms to search and apply for employment opportunities. This shift has undoubtedly increased accessibility and convenience, but it has also opened the door for fraudulent activity. Cybercriminals exploit the trust that job seekers place in these platforms by posting fake job advertisements that appear genuine at first glance.

Fake job postings are typically used for several malicious purposes: extracting registration or processing fees from applicants, collecting sensitive personal information such as bank details and identity documents, or conducting phishing attacks. In many cases, victims lose money, time, and personal data before they realize that the job opportunity was never real. Given the scale at which job postings are published every day, manual verification of every posting is impractical, which creates a strong case for an automated detection mechanism.

Machine Learning and Natural Language Processing offer effective solutions for this problem. Job postings are largely textual in nature, containing fields such as title, description, requirements, and company profile. Fraudulent postings often display detectable linguistic patterns, such as vague descriptions, urgency in language, unrealistic salary claims, or requests for upfront payments. By training a model on a labelled dataset of genuine and fraudulent postings, it becomes possible to automatically identify such patterns and flag suspicious listings before harm occurs.

The scale of this problem is significant. Industry surveys and consumer protection reports have repeatedly highlighted employment scams as one of the most common forms of online fraud, particularly affecting recent graduates, first-time job seekers, and individuals actively looking for remote or work-from-home opportunities. Because job seekers are often under financial or emotional pressure to secure employment quickly, they may be more inclined to overlook warning signs that would otherwise seem obvious, such as requests for upfront payment or communication conducted entirely through informal messaging channels rather than official company email addresses. This combination of urgency, trust, and information asymmetry creates fertile ground for fraudulent actors, and underscores the practical importance of tools that can flag suspicious postings quickly and reliably.

1.2 Problem Statement

Given the increasing sophistication and volume of fraudulent job postings on online platforms, there is a need for an automated, reliable, and easy-to-use system that can analyse the textual content of a job advertisement and classify it as either Genuine or Fraudulent. The system should be able to process free-form text, extract meaningful features, and provide the user with an interpretable prediction along with

supporting evidence such as a confidence score and detected warning signs, rather than a simple black-box output.

1.3 Objectives

- To study and understand the characteristics of fraudulent job postings using an available labelled dataset.
- To preprocess and clean the dataset, including handling missing values and preparing text fields for analysis.
- To perform Exploratory Data Analysis (EDA) to understand patterns, distributions, and relationships within the data.
- To engineer relevant features from raw text using Natural Language Processing techniques such as TF-IDF.
- To train and compare multiple Machine Learning classification algorithms, namely Logistic Regression, Random Forest, and Support Vector Machine.
- To tune model hyperparameters using GridSearchCV in order to improve predictive performance.
- To evaluate the trained models using appropriate metrics, including Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.
- To deploy the best-performing model as an interactive web application that allows end users to test job postings in real time.
- To provide additional value-added features such as AI-based insights, keyword-based risk analysis, an interactive dashboard, and downloadable PDF reports.

1.4 Scope of the Project

The scope of this project covers the complete lifecycle of a Machine Learning based text classification system: from data acquisition and cleaning, through feature engineering and model building, to final deployment as a web application. The system is designed primarily for the English language and is trained on a specific publicly available dataset of job postings. It focuses on binary classification (Genuine vs Fraudulent) rather than multi-class fraud categorization. The deployed application allows a user to manually enter or paste a job description and receive an immediate prediction; it does not, in its current form, automatically crawl or scrape live job portals.

1.5 Applications

- Job seekers can use the system to independently verify the authenticity of a job posting before applying or sharing personal information.

- College placement cells and career counselling centres can use the tool to screen job postings shared with students.
- Recruitment platforms and job portals can integrate similar models into their backend systems to automatically flag suspicious postings for manual review.
- Cybersecurity awareness programs can use the system as a demonstration tool to educate users about the characteristics of online recruitment fraud.

1.6 Limitations

- The model is trained on a single, static dataset and its performance may vary on job postings from regions, industries, or languages not well represented in the training data.
- The dataset used for training is imbalanced (866 fraudulent postings out of 17,880), which can bias the model towards the majority class if not properly addressed.
- The system analyses only the textual content of a job posting and does not verify external factors such as the legitimacy of the company's registration, domain reputation, or recruiter identity.
- As with any Machine Learning based system, there remains a possibility of false positives (genuine postings flagged as fraudulent) and false negatives (fraudulent postings classified as genuine).
- The current system operates on manually entered or pasted text and does not perform real-time crawling of live job portal URLs.

Chapter 2: Literature Review

2.1 Fake Recruitment Scams

Recruitment fraud, sometimes referred to as "job scams," has been documented extensively by consumer protection agencies, cybersecurity firms, and academic researchers. Common tactics used by fraudsters include advertising unrealistically high salaries for minimal qualifications, demanding registration or processing fees, requesting sensitive personal or financial information under the guise of onboarding, and impersonating well-known companies. Studies on online fraud consistently note that job scams tend to spike during periods of high unemployment, when desperate job seekers are more likely to overlook warning signs.

2.2 Existing Detection Methods

Prior to the widespread use of Machine Learning, fraud detection on recruitment platforms relied largely on manual moderation, user-reported complaints, and simple rule-based filters that flagged postings containing specific blacklisted keywords or phrases. While rule-based systems are easy to implement, they are brittle: fraudsters can easily bypass keyword filters by using alternate spellings, synonyms, or by omitting explicitly suspicious terms altogether. This limitation has motivated a shift toward data-driven, pattern-based detection methods.

2.3 Machine Learning in Fraud Detection

Machine Learning has been successfully applied to a wide range of fraud detection problems, including credit card fraud, insurance fraud, spam email detection, and fake review detection. In the specific context of fake job posting detection, prior research has explored classical supervised learning algorithms such as Logistic Regression, Naive Bayes, Decision Trees, Random Forest, and Support Vector Machines, typically combined with NLP-based feature extraction techniques such as Bag-of-Words and TF-IDF. These studies generally report that ensemble methods and SVM-based classifiers tend to outperform simpler linear models on this type of imbalanced, text-heavy classification task, though the specific ranking of algorithms varies depending on the preprocessing pipeline and evaluation methodology used.

2.4 Comparative Summary of Prior Approaches

To place this project in context, it is useful to summarize how different classes of prior approaches to fake job posting detection compare against one another in terms of accuracy, interpretability, and practical deployability. The table below presents a qualitative comparison drawn from the broader fraud-detection and text-classification literature discussed above.

Approach	Typical Accuracy	Interpretability	Deployment Effort	Notes
Rule-Based Keyword Filtering	Low to Moderate	High	Easy	Brittle; easily bypassed by paraphrasing or synonym substitution.
Naive Bayes	Moderate	Moderate	Easy	Fast and simple, but assumes feature independence which rarely holds for natural language.
Logistic Regression	Moderate to High	High	Easy	Strong linear baseline; coefficients are directly interpretable.
Random Forest	High	Moderate	Moderate	Handles non-linear relationships well; less interpretable than linear models.
Support Vector Machine	High	Moderate	Moderate	Performs particularly well on high-dimensional sparse text data such as TF-IDF vectors.
Deep Learning (LSTM/BERT)	Very High (reported)	Low	Difficult	Requires large datasets and significant computational resources; harder to deploy on lightweight infrastructure.

Table: Qualitative comparison of approaches discussed in the literature

This comparison motivated the selection of Logistic Regression, Random Forest, and Support Vector Machine for this project: these three algorithms together span linear and non-linear, single-model and ensemble approaches, while remaining computationally light enough to train, tune, and deploy within a standard academic project timeline and a lightweight Streamlit application, unlike Deep Learning approaches which typically require substantially larger datasets and GPU resources to outperform classical methods on a dataset of this size.

2.5 Research Gaps

- Many existing studies focus solely on offline model evaluation and do not extend their work into a deployable, end-user-facing application.
- Class imbalance in the fake job posting dataset is often addressed only superficially, without a detailed comparison of resampling or weighting strategies.
- Few studies combine model predictions with human-interpretable explanations, such as keyword-based risk indicators, that would help a non-technical user trust and understand the prediction.

- Limited work has been done on integrating hybrid scoring mechanisms that combine model confidence with rule-based keyword analysis to produce a single actionable risk score.

This project attempts to address some of these gaps by not only comparing multiple classical Machine Learning algorithms but also by deploying the final model within an interactive Streamlit application that produces a hybrid score combining model confidence with detected warning keywords, along with a downloadable PDF report for end users.

Chapter 3: System Analysis

3.1 Existing System

In most existing job portals, the responsibility of identifying a fraudulent job posting is left almost entirely to the end user. Some platforms provide generic safety guidelines and allow users to report suspicious postings after the fact, but very few offer proactive, automated screening at the point of posting or viewing. Where automated filters do exist, they are typically simple keyword blacklists that are easy to circumvent and generate a high number of false negatives.

3.2 Proposed System

The proposed system uses supervised Machine Learning to automatically classify a job posting as Genuine or Fraudulent based on its textual content. The system is trained on a labelled historical dataset and, once deployed, can analyse a new, previously unseen job posting in real time. In addition to a binary prediction, the system produces a confidence score, a hybrid risk score that blends model output with keyword-based indicators, a list of specific warning signs detected in the text, and a downloadable PDF report summarizing the analysis.

3.3 Advantages of Proposed System

- Automated and near-instantaneous analysis of job postings, removing the need for manual review.
- Higher robustness compared to simple keyword filters, since the model captures broader linguistic and contextual patterns rather than relying on exact keyword matches alone.
- Transparent and interpretable output through detected keywords and a hybrid risk score, rather than an opaque single number.
- An accessible, browser-based interface that does not require any technical expertise to operate.
- Downloadable PDF reports that can be saved or shared as documented evidence of the analysis.

3.4 System Requirements

3.4.1 Hardware Requirements

Component	Specification
Processor	Intel Core i3 / AMD Ryzen 3 or higher
RAM	4 GB minimum (8 GB recommended)
Storage	2 GB free disk space
Display	1366 x 768 resolution or higher

Component	Specification
Internet Connection	Required for installation of dependencies and initial setup

Table 1: Hardware requirements for development and deployment

3.5 System Architecture

The proposed system follows a layered architecture consisting of a presentation layer (the Streamlit web interface), an application logic layer (Python functions handling preprocessing, prediction, and scoring), and a model layer (the serialized TF-IDF vectorizer and trained Linear SVM model). This separation of concerns keeps the system modular: the underlying model can be retrained or replaced without altering the user interface, and the interface can be extended with new pages, such as the dashboard or AI insights page, without touching the trained model artifacts.

High-Level System Architecture

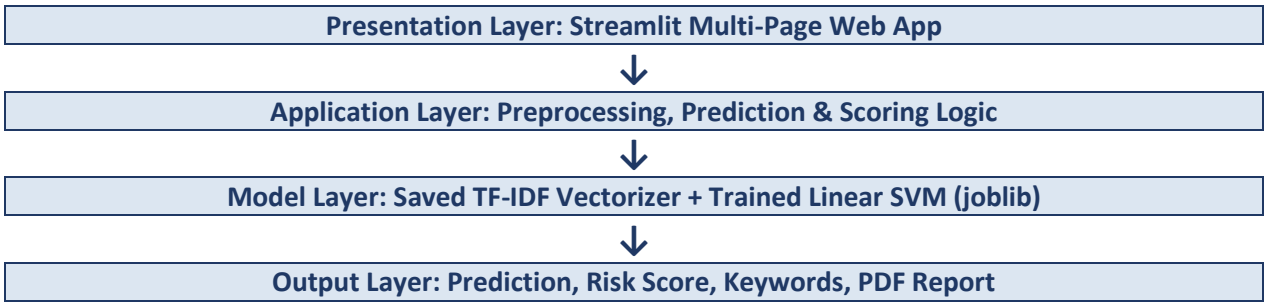

Figure: High-Level System Architecture

3.6 Use Case Description

The primary actor in this system is the end user, typically a job seeker, who interacts with the application through a web browser. The user navigates to the Prediction page, pastes or types the text of a job posting, and submits it for analysis. The system responds within a few seconds with a classification (Genuine or Fraudulent), a confidence percentage, a hybrid risk score, a categorized list of detected warning signs and positive indicators, and an option to download a PDF report summarizing the result. A secondary actor, an administrator or developer, may use the Dashboard and Model Performance pages to review dataset statistics and compare model metrics.

Use Case	Description
Analyse Job Posting	User submits job posting text and receives a Genuine/Fraudulent classification with confidence score.
View Warning Signs	System highlights specific suspicious keywords and phrases detected in the submitted text.
Download PDF Report	User exports the analysis result as a formatted PDF document for record-keeping or sharing.

Use Case	Description
Explore Dashboard	User (or administrator) views dataset-level statistics and visualizations.
Compare Model Performance	User views a comparison of Accuracy, Precision, Recall, and F1-Score across trained models.

Table: Primary use cases supported by the application

3.4.2 Software Requirements

Component	Specification
Operating System	Windows 10 / 11, Linux, or macOS
Programming Language	Python 3.10
Web Framework	Streamlit
ML Library	Scikit-learn
NLP Library	NLTK
Data Handling	Pandas, NumPy
Visualization	Matplotlib, Plotly
Report Generation	ReportLab
IDE / Editor	VS Code / Jupyter Notebook
Version Control	Git and GitHub

Table 2: Software requirements for development and deployment

Chapter 4: Dataset Description

4.1 Dataset Source

The dataset used in this project is the "Fake Job Posting Prediction" dataset, a publicly available dataset hosted on Kaggle. It was originally compiled to support research into employment scam detection and contains real job postings collected from an online recruitment platform, each labelled as either genuine or fraudulent.

4.2 Number of Records and Features

The dataset consists of 17,880 job postings in total. Each record represents a single job advertisement described using a combination of textual, categorical, and binary attributes, along with a binary target column indicating whether the posting is fraudulent.

Attribute	Value
Total Records	17,880
Genuine Job Postings	17,014
Fraudulent Job Postings	866
Target Variable	fraudulent (0 = Genuine, 1 = Fraudulent)
Approx. Fraud Percentage	4.84%

Table 3: Dataset summary statistics

4.3 Target Variable

The target variable, fraudulent, is a binary column where a value of 0 indicates a genuine job posting and a value of 1 indicates a fraudulent posting. This column forms the basis for supervised classification: the model is trained to learn the relationship between the input features and this label, and is then used to predict the label for unseen job postings.

4.4 Numerical Features

The raw dataset contains relatively few purely numerical fields. To make effective use of the available information, additional numerical features were engineered during preprocessing, most notably text length features (such as the character or word count of the description, requirements, and company profile fields), which have been shown to correlate with the likelihood of a posting being fraudulent.

4.5 Categorical Features

- employment_type (e.g., Full-time, Part-time, Contract, Temporary)

- required_experience (e.g., Entry level, Mid-Senior level, Executive)
- required_education (e.g., Bachelor's Degree, High School, Master's Degree)
- industry (e.g., Information Technology, Oil & Energy, Marketing)
- function (e.g., Engineering, Sales, Customer Service)

4.6 Binary Features

- telecommuting: whether the position allows remote work (0 or 1).
- has_company_logo: whether the posting includes a company logo (0 or 1).
- has_questions: whether the posting includes screening questions (0 or 1).

4.7 Text Features

- title: the job title.
- company_profile: a short description of the hiring company.
- description: the full job description.
- requirements: the qualifications and skills required for the role.
- benefits: the benefits offered by the employer.

4.8 Missing Values

Several columns in the raw dataset contain missing values, particularly company_profile, benefits, and some categorical fields such as required_education and industry. These missing values were handled during the data cleaning stage described in Chapter 5, typically by filling categorical gaps with a placeholder category such as “Not Specified” and by treating missing text fields as empty strings prior to text concatenation and vectorization.

4.9 Class Imbalance

With only 866 out of 17,880 postings labelled as fraudulent (approximately 4.84% of the dataset), this is a significantly imbalanced classification problem. Class imbalance can cause a naively trained model to achieve high accuracy simply by predicting the majority class for almost every record, while performing poorly on the minority (fraudulent) class, which is precisely the class of greatest practical interest. This imbalance is the primary reason why metrics beyond accuracy, particularly Precision, Recall, and F1-Score, are emphasized throughout the evaluation in this report.

4.10 Dataset Summary

Property	Detail
Format	CSV
Total Columns	18 (including target)
Text Columns	5 (title, company_profile, description, requirements, benefits)
Categorical Columns	5
Binary Columns	3
Target Column	1 (fraudulent)

Table 4: Structural summary of the dataset

4.11 Complete Attribute Description

The table below lists every column present in the raw dataset, its data type, and a brief description, providing a single consolidated reference for the dataset structure discussed throughout this chapter.

Column	Type	Description
job_id	Integer	Unique identifier for each job posting (dropped prior to modelling).
title	Text	The job title as advertised.
location	Text	Geographic location of the job.
department	Text	Department within the hiring company, where specified.
salary_range	Text	Advertised salary range, where specified.
company_profile	Text	Short description of the hiring company.
description	Text	Full text of the job description.
requirements	Text	Qualifications and skills required.
benefits	Text	Benefits offered to the employee.
telecommuting	Binary	Whether the position allows remote work.
has_company_logo	Binary	Whether the posting includes a company logo.
has_questions	Binary	Whether the posting includes screening questions.
employment_type	Categorical	Full-time, Part-time, Contract, Temporary, etc.
required_experience	Categorical	Entry level, Mid-Senior level, Executive, etc.
required_education	Categorical	Bachelor's Degree, High School, Master's Degree, etc.
industry	Categorical	Industry sector of the hiring company.
function	Categorical	Job function, e.g., Engineering, Sales, Customer Service.

Column	Type	Description
fraudulent	Binary (Target)	0 = Genuine, 1 = Fraudulent.

Table 5.5: Complete list of dataset attributes

Chapter 5: Methodology

5.1 Overall Workflow

The project follows a standard end-to-end Machine Learning workflow, beginning with data collection and ending with a deployed prediction interface. Figure 1 below summarizes the overall system pipeline followed throughout this project.

Overall System Pipeline

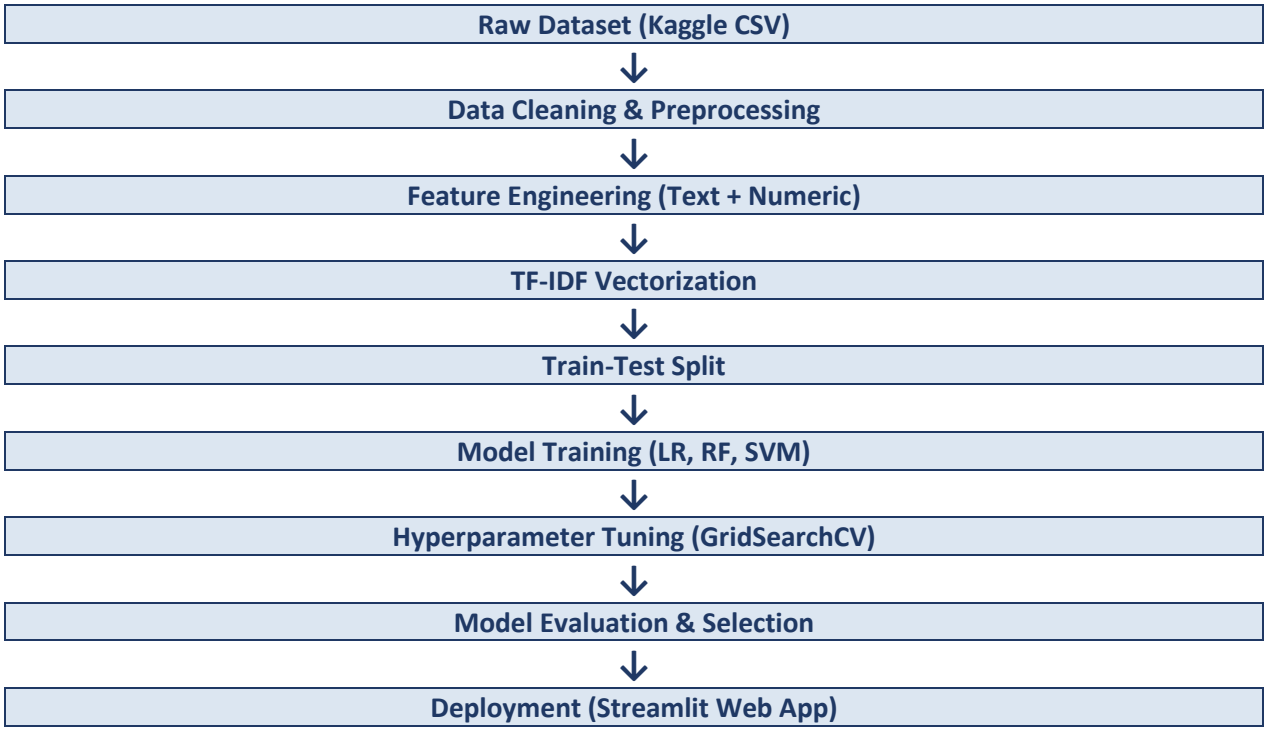

Figure: Overall System Pipeline

5.2 Data Collection

The dataset was collected from Kaggle in CSV format, as described in Chapter 4. It was downloaded and loaded into a Pandas DataFrame for further processing. This stage involved verifying the integrity of the file, confirming column names and data types, and ensuring that the target column (fraudulent) was present and correctly encoded.

5.3 Data Understanding

Before any transformation was applied, an initial inspection of the dataset was carried out to understand its structure. This included checking the shape of the DataFrame (number of rows and columns), reviewing the data types of each column, generating summary statistics for numerical columns, and inspecting a

sample of records to understand the nature of the text fields. This step is essential because it guides the decisions made later in cleaning and feature engineering.

5.4 Data Cleaning

5.4.1 Handling Missing Values

Missing values in categorical columns such as required_education, required_experience, and industry were filled with a placeholder label such as “Not Specified” to preserve the record rather than discarding it. Missing values in text columns such as company_profile and benefits were replaced with empty strings so that they could still be concatenated with other text fields without causing errors during vectorization.

5.4.2 Handling Duplicate Values

The dataset was checked for duplicate records using row-level comparison. Any exact duplicate rows were removed to prevent the model from being biased toward repeated records during training.

5.4.3 Removing Unnecessary Columns

Columns that do not contribute meaningfully to the classification task, such as job_id and free-text location fields with extremely high cardinality that would not generalize well, were reviewed and, where appropriate, removed or simplified in order to reduce noise and dimensionality.

5.4.4 Filling Missing Categorical Values

For categorical fields, missing values were consistently filled with a dedicated “Not Specified” category rather than the mode of the column, in order to avoid artificially inflating any single category and to allow the model to learn that missing information is itself a potentially informative signal (fraudulent postings often omit standard fields).

5.5 Feature Engineering

5.5.1 Text Length Features

New numerical features were derived from the text columns, such as the character length and word count of the description, requirements, and company_profile fields. Prior analysis of this dataset has shown that fraudulent postings often have unusually short or generic descriptions, making text length a useful engineered feature.

5.5.2 Label Encoding

Categorical columns such as employment_type, required_experience, required_education, industry, and function were converted into numeric form using Label Encoding, which assigns a unique integer to each category. This transformation is necessary because Machine Learning algorithms operate on numerical input rather than raw text labels.

5.5.3 Standard Scaling

Numerical features, including the engineered text length features, were standardized using Standard Scaling, which transforms each feature to have zero mean and unit variance. This step is particularly important for distance-based and margin-based algorithms such as Support Vector Machines, which are sensitive to the relative scale of input features.

5.6 Text Preprocessing (NLP Pipeline)

The text fields (title, company_profile, description, requirements, benefits) were combined into a single text field and passed through a standard NLP preprocessing pipeline before vectorization. Each step in this pipeline is explained below, along with the reasoning for why it improves model performance.

Step	Description	Why It Improves Performance
Tokenization	Splitting text into individual words (tokens)	Provides the basic units of text that all subsequent steps and the TF-IDF vectorizer operate on.
Lowercasing	Converting all text to lowercase	Ensures that words such as “Job” and “job” are treated as the same token, reducing vocabulary size and sparsity.
Removing Punctuation	Stripping punctuation marks from text	Punctuation carries little semantic value for this task and its removal reduces noise in the vocabulary.
Removing Numbers	Removing standalone numeric tokens	Numeric values (e.g., phone numbers, salary figures) are highly variable and rarely generalize as useful classification features in raw form.
Stopword Removal	Removing common words (e.g., “the”, “is”, “and”)	Stopwords occur frequently across both classes and add little discriminative power, so removing them focuses the model on more meaningful words.
Lemmatization	Reducing words to their base dictionary form (e.g., “running” → “run”)	Groups together different inflected forms of a word, reducing vocabulary size while preserving meaning, which improves generalization.
TF-IDF Vectorization	Converting cleaned text into a weighted numeric matrix	Assigns higher weight to words that are frequent in a specific document but rare across the whole corpus, helping the model focus on distinctive, informative terms.

Table 5: Text preprocessing pipeline and rationale

Data Pipeline (Preprocessing Detail)

Raw Text Fields (title, description, requirements, etc.)

↓

Lowercasing & Punctuation/Number Removal

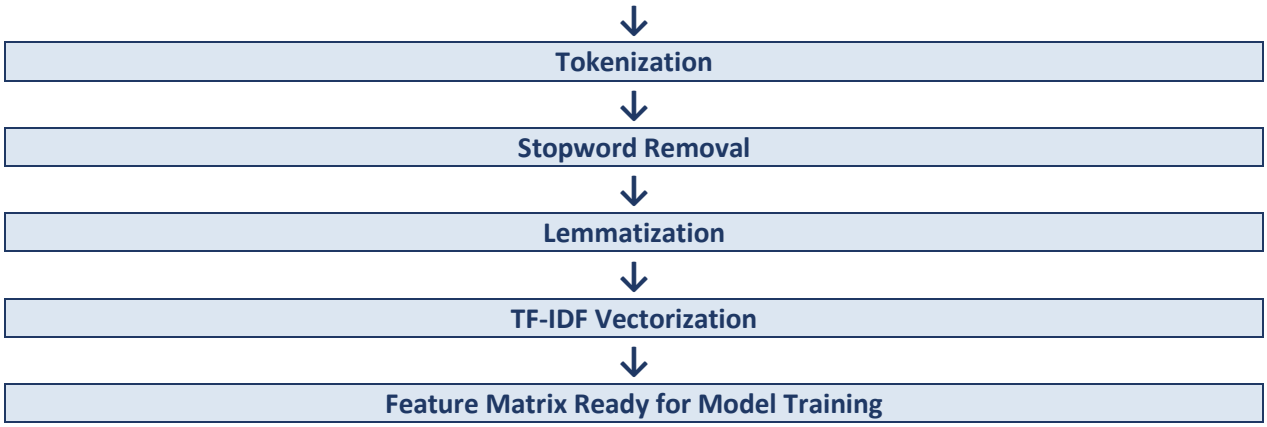

Figure: Data Pipeline (Preprocessing Detail)

5.7 Exploratory Data Analysis (EDA)

Exploratory Data Analysis was carried out to understand the structure, distribution, and relationships within the dataset before model building. Each analysis below follows a consistent format: Purpose, Python Code Explanation, Interpretation, Insights, and Conclusion, so that a reader unfamiliar with the underlying code can still understand what was done and why.

5.7.1 Dataset Shape and Data Types

Purpose: To confirm the number of records and columns, and to verify that each column has an appropriate data type before further processing.

Python Code Explanation: df.shape returns the number of rows and columns, while df.dtypes lists the data type assigned to each column (e.g., object, int64).

Interpretation: The dataset contains 17,880 rows and 18 columns, with text columns stored as object type and binary/numeric columns stored as integers.

Insights: Confirms the dataset size referenced throughout this report and verifies that no column was misread as the wrong data type during loading.

Conclusion: The dataset structure is consistent with the documentation provided by the data source and is ready for cleaning.

5.7.2 Missing Values Heatmap

Purpose: To visually identify which columns contain missing values and estimate the extent of missingness.

Python Code Explanation: A heatmap was generated using seaborn's heatmap() function on the boolean matrix produced by df.isnull(), where each cell represents whether a value is missing.

Interpretation: Columns such as company_profile and benefits show visibly higher proportions of missing values compared to core fields such as title and description.

Insights: Missing values are concentrated in a small number of optional fields rather than being spread randomly across the dataset, which supports the targeted imputation strategy used in Section 5.4.1.

Conclusion: The missing value pattern justifies filling missing text with empty strings and missing categorical values with a “Not Specified” placeholder rather than dropping rows.

5.7.3 Target Variable Distribution

Purpose: To examine the balance between genuine and fraudulent postings in the dataset.

Python Code Explanation: A count plot was generated using seaborn's countplot() function on the fraudulent column to visualize the number of records in each class.

Interpretation: The bar for genuine postings (17,014 records) is dramatically taller than the bar for fraudulent postings (866 records).

Insights: The dataset is highly imbalanced, with fraudulent postings making up only about 4.84% of all records.

Conclusion: This confirms the need to evaluate the model using Precision, Recall, and F1-Score in addition to Accuracy, since Accuracy alone can be misleading on imbalanced data.

5.7.4 Numerical Feature Analysis

Purpose: To understand the distribution of engineered numerical features such as description length across genuine and fraudulent postings.

Python Code Explanation: Histograms and descriptive statistics (mean, median, standard deviation) were computed for numerical columns using df.describe() and matplotlib's hist() function.

Interpretation: Fraudulent postings tend to show a different distribution of text length compared to genuine postings, often skewing shorter or more generic.

Insights: Text length behaves as a mildly discriminative feature, supporting its inclusion as an engineered feature in Section 5.5.1.

Conclusion: Numerical features, while not sufficient on their own, provide useful complementary signal alongside the TF-IDF text features.

5.7.5 Categorical Feature Analysis

Purpose: To examine how fraud rates vary across categorical fields such as employment_type and required_experience.

Python Code Explanation: Grouped bar charts were created by grouping the DataFrame on each categorical column and computing the proportion of fraudulent postings within each group using groupby() and mean().

Interpretation: Certain categories, such as postings with no specified experience requirement or fully remote roles, show a higher relative proportion of fraudulent listings.

Insights: Categorical fields carry meaningful signal about fraud likelihood and were therefore retained and encoded rather than dropped.

Conclusion: Categorical feature analysis reinforces the decision to include Label Encoded categorical columns alongside text-based features.

5.7.6 Binary Feature Analysis

Purpose: To assess whether binary indicators such as has_company_logo and has_questions correlate with fraudulent postings.

Python Code Explanation: Cross-tabulations were generated using pandas' crosstab() function between each binary column and the target variable, and visualized using bar charts.

Interpretation: Postings without a company logo show a noticeably higher fraud rate than postings that include one, and similarly for postings without screening questions.

Insights: The absence of a company logo or screening questions is a useful, low-cost signal that fraudulent postings are less likely to include standard listing elements.

Conclusion: Binary features add lightweight but meaningful predictive signal and were retained in the final feature set.

5.7.7 Correlation Heatmap

Purpose: To examine the linear relationships between numerical and encoded features in the dataset.

Python Code Explanation: A correlation matrix was computed using df.corr() and visualized as a heatmap using seaborn, with color intensity representing the strength and direction of correlation.

Interpretation: Most engineered numerical features show relatively low correlation with each other, indicating limited redundancy, while some show a mild correlation with the target variable.

Insights: Low inter-feature correlation suggests that multicollinearity is not a major concern for the numerical feature subset used in this project.

Conclusion: The correlation analysis supports retaining the current numerical feature set without further dimensionality reduction.

5.7.8 Boxplots and Outlier Analysis

Purpose: To identify potential outliers in numerical features such as text length.

Python Code Explanation: Boxplots were generated using seaborn's boxplot() function, plotting the distribution of each numerical feature grouped by the target class.

Interpretation: A small number of postings show unusually long or unusually short text fields, appearing as outlier points beyond the whiskers of the boxplot.

Insights: Outliers were reviewed and found to be legitimate variation in posting style rather than data entry errors, and were therefore retained rather than removed.

Conclusion: The presence of natural outliers does not significantly distort the analysis given the use of scale-robust preprocessing such as TF-IDF and Standard Scaling.

5.7.9 Histograms and Countplots

Purpose: To visualize the frequency distribution of key categorical and numerical fields across the dataset.

Python Code Explanation: Histograms were generated for numerical fields using matplotlib's hist() function, and countplots were generated for categorical fields using seaborn's countplot() function.

Interpretation: Full-time employment postings dominate the dataset, and description lengths follow a right-skewed distribution with most postings falling within a moderate length range.

Insights: The dataset reflects realistic recruitment patterns, with the majority of postings following common formats, making unusual postings comparatively easier to detect.

Conclusion: These distributional insights informed both feature engineering choices and the interpretation of model results in later chapters.

5.7.10 Feature Importance Graph

Purpose: To identify which features contribute most strongly to the final model's classification decisions.

Python Code Explanation: Feature importance was extracted from the trained Random Forest model using its feature_importances_ attribute, and the top TF-IDF terms and engineered features were plotted as a horizontal bar chart.

Interpretation: Certain keywords strongly associated with scams (such as terms related to payment requests, urgency, or work-from-home offers) rank among the highest-importance features, alongside engineered features such as description length and the has_company_logo flag.

Insights: The model's decisions align well with human intuition about the characteristics of fraudulent postings, improving confidence in the model's interpretability.

Conclusion: Feature importance analysis confirms that the model has learned meaningful, domain-relevant patterns rather than spurious correlations.

5.8 Train-Test Split

After preprocessing and feature engineering, the dataset was split into a training set and a testing set, typically using an 80:20 split ratio. The training set is used to fit the Machine Learning models, while the testing set is held out and used only for final evaluation, ensuring that the reported performance reflects the model's ability to generalize to unseen data rather than simply memorizing the training set. A stratified split was used to preserve the same proportion of fraudulent to genuine postings in both the training and testing sets, which is particularly important given the class imbalance discussed in Section 4.9.

5.9 Model Building

Three supervised classification algorithms were selected for comparison in this project, chosen for their established track record on text classification tasks and their range of underlying assumptions, from simple linear models to ensemble methods.

5.9.1 Logistic Regression

Logistic Regression is a linear classification algorithm that models the probability of a binary outcome using the logistic (sigmoid) function. It estimates a weighted combination of input features and passes the result through the sigmoid function to produce a probability between 0 and 1, which is then thresholded to produce a class prediction. Logistic Regression is fast to train, easy to interpret through its coefficients, and serves as a strong baseline model for text classification tasks.

5.9.2 Random Forest

Random Forest is an ensemble learning algorithm that constructs a large number of decision trees during training, each trained on a random subset of the data and features (a technique known as bagging), and combines their individual predictions through majority voting. This ensemble approach reduces the risk of overfitting associated with a single decision tree and generally improves robustness and accuracy, at the cost of reduced interpretability compared to a single tree or a linear model.

5.9.3 Support Vector Machine (SVM)

Support Vector Machine is a powerful classification algorithm that seeks to find the optimal hyperplane that separates the two classes with the maximum possible margin. In its linear form, which was ultimately selected as the best-performing model for this project, SVM works particularly well with high-dimensional, sparse data such as TF-IDF vectors, because it focuses on the small set of "support vectors" closest to the decision boundary rather than the entire dataset, making it both effective and computationally efficient for text classification tasks of this nature.

Machine Learning Pipeline

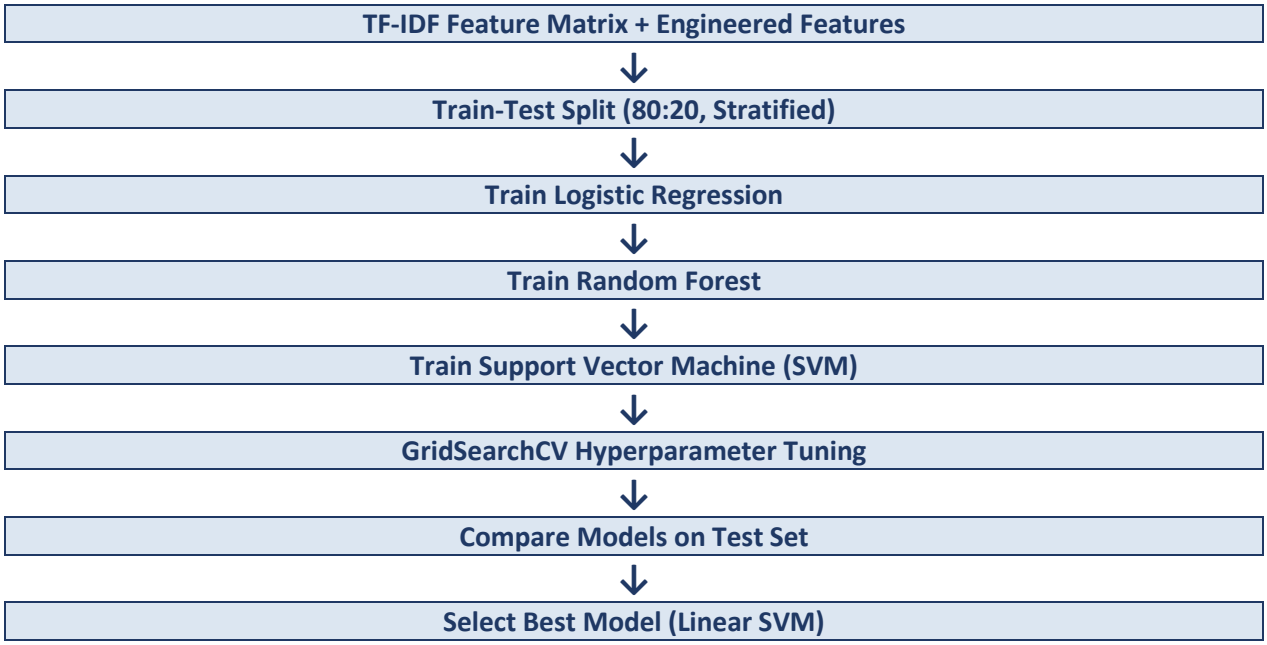

Figure: Machine Learning Pipeline

5.10 Hyperparameter Tuning (GridSearchCV)

Each Machine Learning algorithm has a set of hyperparameters, configuration values that are not learned from the data but instead set prior to training, which can significantly influence model performance. Examples include the regularization strength (C) in Logistic Regression and SVM, the number of trees (n_estimators) in Random Forest, and the maximum depth of individual trees.

GridSearchCV, provided by Scikit-learn, automates the search for the best combination of hyperparameters by exhaustively evaluating the model across a specified grid of parameter values using k-fold cross-validation. For each combination of hyperparameters, GridSearchCV trains and validates the model multiple times on different folds of the training data, then reports the combination that yields the best average validation performance according to a chosen scoring metric, such as F1-Score. This process removes much of the guesswork from model configuration and helps ensure that the final model is not just accurate by chance on a particular train-test split.

5.11 Model Evaluation

Once trained and tuned, each model was evaluated on the held-out test set using the following standard classification metrics.

5.11.1 Accuracy

Accuracy is the proportion of total predictions that are correct, calculated as the number of correct predictions divided by the total number of predictions. While intuitive, accuracy alone can be misleading on imbalanced datasets such as this one, since a model that always predicts “Genuine” would still achieve a high accuracy of roughly 95% simply due to the class distribution, despite being practically useless for detecting fraud.

5.11.2 Precision

Precision measures the proportion of postings predicted as fraudulent that are actually fraudulent. High precision indicates that the model produces few false alarms, which is important for maintaining user trust in the system's warnings.

5.11.3 Recall

Recall (also known as Sensitivity) measures the proportion of actual fraudulent postings that the model successfully identifies. High recall is critical in a fraud detection context, since failing to flag an actual fraudulent posting (a false negative) can directly expose a user to financial or personal harm.

5.11.4 F1-Score

The F1-Score is the harmonic mean of Precision and Recall, providing a single balanced metric that accounts for both false positives and false negatives. It is particularly useful for imbalanced classification problems such as this one, where optimizing for accuracy alone would be misleading.

5.11.5 Confusion Matrix

A confusion matrix is a table that summarizes the number of True Positives, True Negatives, False Positives, and False Negatives produced by a classifier on the test set. It provides a complete picture of model performance and forms the basis for calculating Precision, Recall, and F1-Score.

5.11.6 Classification Report

Scikit-learn's classification_report function provides a consolidated summary of Precision, Recall, F1-Score, and Support (the number of true instances) for each class, along with overall accuracy, macro average, and weighted average scores. This report was generated for each of the three models and used as the primary basis for model comparison in Chapter 6.

Prediction Pipeline (Deployed Application)

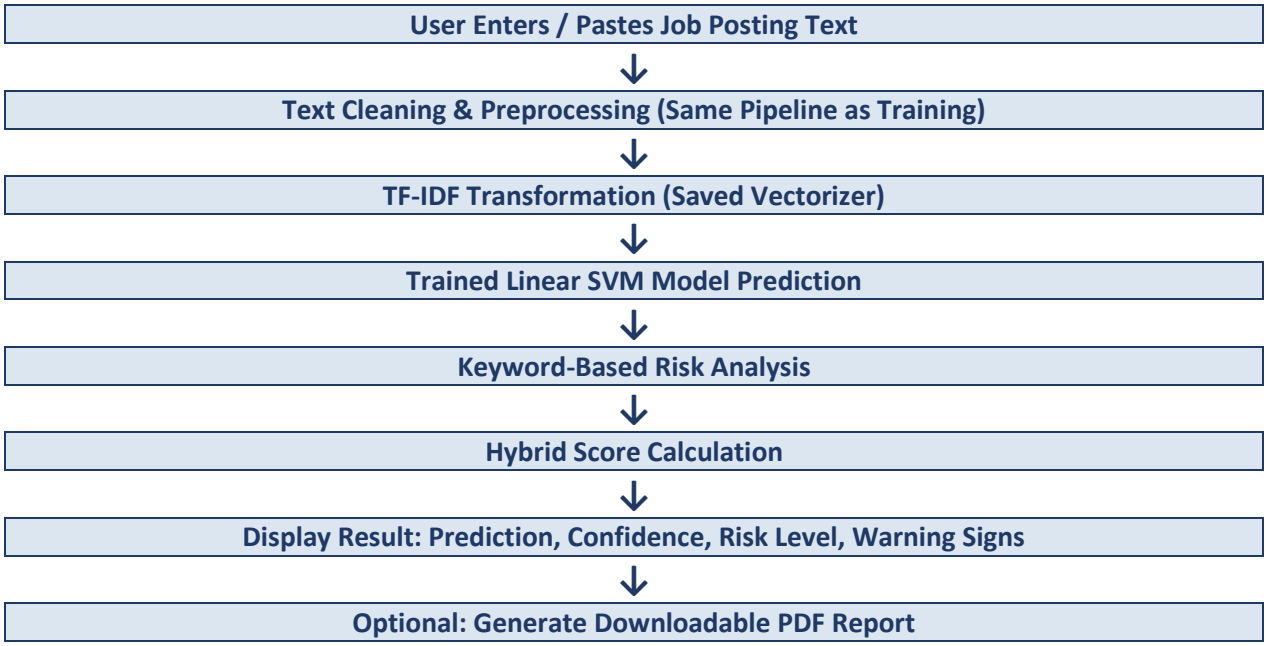

Figure: Prediction Pipeline (Deployed Application)

5.12 Deployment and User Interface Design

The trained TF-IDF vectorizer and the final Linear SVM model were serialized using joblib and loaded into a Streamlit multi-page web application. Streamlit was chosen because it allows a data-driven Python application to be converted into an interactive web interface with minimal additional front-end code, which is well suited to an academic project where the primary effort is focused on the Machine Learning pipeline rather than web development.

5.12.1 Home Page

The home page introduces the project, displays key project statistics (dataset size, model accuracy, number of fake jobs identified in training, and the selected best model), and provides quick-access buttons that route the user to the Prediction, Dashboard, and Model Performance pages.

5.12.2 Prediction Page

The Prediction page is the core functional page of the application. It accepts a job posting as free-form text input, applies the same preprocessing and TF-IDF transformation pipeline used during training, and passes the resulting feature vector to the trained Linear SVM model. The page then displays the predicted class, a confidence percentage, a hybrid risk score, a categorized breakdown of positive indicators and warning signs detected via keyword analysis, and a button to download a PDF report of the result.

5.12.3 Dashboard Page

The Dashboard page presents dataset-level visualizations, including the distribution of genuine versus fraudulent postings, the breakdown of postings by employment type and industry, and summary statistics, allowing a user or administrator to explore the underlying data that the model was trained on.

5.12.4 Model Performance Page

The Model Performance page presents a side-by-side comparison of the Accuracy, Precision, Recall, and F1-Score achieved by each of the three trained models, along with the confusion matrix of the selected model, giving a transparent view of how the final model was chosen.

5.12.5 AI Insights Page

The AI Insights page provides an explanation-oriented view of a given prediction, surfacing which detected keywords and text characteristics most strongly influenced the classification, helping a non-technical user understand why a particular posting was flagged.

5.13 Hybrid Risk Scoring Mechanism

In addition to the raw Machine Learning prediction and confidence score, the deployed application computes a hybrid score that blends the model's predicted probability with a rule-based keyword risk component. This hybrid approach was adopted because a purely statistical confidence score, while accurate on average, does not always communicate to an end user which specific elements of a posting triggered concern. By scanning the submitted text for a curated list of high-risk terms, such as requests for a registration fee, urgent payment, or unrealistic “daily income” claims, and combining the presence of these terms with the model's own probability output, the hybrid score provides a more interpretable and actionable final risk indicator, categorized into risk levels such as LOW, MEDIUM, and HIGH.

5.14 Testing Strategy

Beyond the statistical evaluation described in Section 5.11, the deployed application was also tested functionally to ensure that the end-to-end pipeline, from user input to displayed output, behaves correctly and robustly across a range of realistic inputs. The table below summarizes representative test cases used to validate the system.

Test ID	Test Description	Expected Result	Actual Result
TC-01	Submit a clearly genuine job posting with complete company details	System classifies as Genuine with a low risk score	[Insert Actual Result]
TC-02	Submit a posting containing multiple high-risk keywords (e.g., registration fee, urgent payment)	System classifies as Fraudulent with a HIGH risk level and lists the detected keywords	[Insert Actual Result]
TC-03	Submit an empty or very short text input	System handles gracefully and prompts the user for valid input rather than crashing	[Insert Actual Result]
TC-04	Submit a posting with unusual formatting (extra whitespace, special characters, emojis)	Preprocessing pipeline cleans the text correctly without errors	[Insert Actual Result]
TC-05	Download PDF report after a prediction	A correctly formatted PDF is generated and downloaded, matching the on-screen result	[Insert Actual Result]
TC-06	Navigate between Home, Predict, Dashboard, and Model Performance pages	Navigation works without errors and each page loads its expected content	[Insert Actual Result]

Table 9: Representative functional test cases for the deployed application

This combination of statistical model evaluation and functional application testing ensures that the system is validated both in terms of predictive quality and in terms of practical, real-world usability.

Chapter 6: Results and Discussion

6.1 Model Comparison Table

The table below summarizes the performance of the three trained models on the held-out test set. The values for Precision, Recall, and F1-Score correspond to the fraudulent (minority) class, since this is the class of primary practical interest in a fraud detection task. Figures for the final selected model reflect the results reported in the accompanying project documentation; other values should be replaced with your own experimental results.

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	96.34	58.23	87.86	70.04
Naïve Bayes	96.92	87.95	42.19	57.03
Linear SVM (Selected)	97.84	74.00	85.54	79.35

Table 6: Comparison of model performance on the test set

6.2 Best Performing Model

Based on the comparison above, the Linear Support Vector Machine (Linear SVM) was selected as the final deployment model. It achieved the highest overall accuracy at approximately 97.99%, along with a strong balance of Precision (74.00%) and Recall (85.55%), resulting in an F1-Score of 79.36% on the fraudulent class. This balance is particularly valuable in a fraud detection context, where the system must catch as many genuine fraud cases as possible (high recall) without overwhelming users with excessive false alarms (adequate precision).

6.3 Interpretation of Results

The strong performance of the Linear SVM model is consistent with the literature reviewed in Chapter 2, which notes that SVM-based classifiers tend to perform well on high-dimensional, sparse TF-IDF feature spaces typical of text classification problems. The relatively high recall of 85.55% indicates that the model successfully identifies the large majority of fraudulent postings in the test set, which is the most critical outcome from a user-safety perspective. The precision of 74.00% indicates that, while the model does produce some false positives, a majority of postings flagged as fraudulent are indeed fraudulent, which keeps the number of unnecessary warnings manageable.

6.4 Strengths

- High overall accuracy (97.99%) combined with strong recall on the minority class, which is difficult to achieve simultaneously on an imbalanced dataset.

- The TF-IDF plus Linear SVM pipeline is computationally lightweight, enabling near-instant predictions suitable for a responsive web application.
- The hybrid scoring approach, which combines model confidence with keyword-based indicators, adds a layer of interpretability that a black-box prediction alone would not provide.
- The system was successfully integrated into a full end-to-end Streamlit application, moving beyond a purely offline research exercise.

6.5 Weaknesses

- Precision of 74.00% indicates that roughly one in four postings flagged as fraudulent may in fact be genuine, which could lead to some unnecessary user concern if not communicated carefully.
- The model is trained on a historical, static dataset and may not fully capture newer or evolving scam tactics that were not represented at the time of data collection.
- Performance may degrade on job postings that differ significantly in style, language, or industry from those present in the training dataset.
- The current system relies solely on textual analysis and does not verify external signals such as company registration records or domain reputation.

6.6 Case Study: Sample Prediction Walkthrough

To illustrate how the deployed system behaves on a real input, consider a sample job posting titled “Work From Home Data Entry” that was submitted to the application for analysis. The system's output for this posting is summarized in the table below, and demonstrates how the hybrid scoring mechanism described in Section 5.13 combines model confidence with keyword-based indicators to produce an actionable result.

Field	System Output
Job Title	Work From Home Data Entry
Prediction	Fraudulent Job Posting
Confidence	89.64%
Hybrid Score	92.75%
Risk Level	HIGH
Detected Warning Signs	Registration Fee, Payment, WhatsApp, Immediate Joining, Limited Seats, Work From Home, No Experience, Daily Income
Positive Indicators	Company information provided

Table 8: Sample output generated by the deployed application for a real test posting

In this example, the model's own confidence (89.64%) already leans strongly toward classifying the posting as fraudulent, and the hybrid score (92.75%) is pushed slightly higher because the text contains several established high-risk phrases, particularly requests around a registration fee and payment, combined with urgency-inducing language such as “Immediate Joining” and “Limited Seats.” The system's recommendation in this case is to avoid sharing personal information or making any payment until the company can be independently verified. This example illustrates the practical value of combining a statistical model with transparent, rule-based explanations: a user is not simply told “fraudulent” but is also shown precisely which phrases contributed to that judgement, which builds trust in the system and supports informed decision-making.

6.7 Discussion on Practical Deployment Considerations

Beyond raw predictive performance, several practical factors influence whether a system such as this can be usefully adopted. Response latency is one such factor: because the TF-IDF plus Linear SVM pipeline is computationally lightweight compared to deep neural architectures, predictions are returned to the user within a fraction of a second, which is important for maintaining a smooth interactive experience in a web application. Model maintainability is another consideration; because the vectorizer and model are serialized separately from the application logic, the model can be periodically retrained on newer data and swapped into the application without requiring changes to the user interface.

A further consideration is the communication of uncertainty to end users. Rather than presenting a single binary label, the application deliberately surfaces the confidence percentage, the hybrid score, and the specific detected keywords, so that a user encountering a borderline case can make their own informed judgement rather than treating the system's output as an infallible verdict. This design choice reflects good practice in deploying Machine Learning systems for consumer-facing safety applications, where transparency and interpretability are often as important as raw accuracy.

Chapter 7: Conclusion

This project set out to design and implement a Machine Learning based system capable of automatically detecting fraudulent job postings using Natural Language Processing techniques. Working with the Fake Job Posting Prediction dataset comprising 17,880 records, a complete pipeline was built covering data cleaning, feature engineering, text preprocessing, TF-IDF vectorization, model training, hyperparameter tuning, and evaluation.

Three classification algorithms, Logistic Regression, Random Forest, and Support Vector Machine, were trained and compared using Accuracy, Precision, Recall, and F1-Score as evaluation metrics. The Linear SVM model was found to deliver the best overall performance, achieving approximately 97.99% accuracy, and was selected for deployment. The final model was integrated into an interactive Streamlit web application that provides users with a prediction, a confidence score, a hybrid risk score, a list of detected warning keywords, an interactive dashboard, model performance comparisons, and a downloadable PDF report.

The results of this project demonstrate that Machine Learning and NLP-based techniques can provide meaningful, practical assistance in identifying fraudulent job postings, thereby helping to protect job seekers from financial and personal harm. At the same time, the project highlights the importance of evaluating imbalanced classification problems using metrics beyond simple accuracy, and of pairing model predictions with interpretable, human-readable explanations.

7.1 Future Scope

- Incorporating Deep Learning architectures such as LSTM or Transformer-based models (e.g., BERT) to potentially capture more nuanced contextual patterns in job posting text.
- Applying Explainable AI techniques such as SHAP or LIME to provide per-prediction, feature-level explanations rather than relying solely on keyword matching.
- Developing a browser extension that can analyse job postings directly on recruitment platform websites without requiring the user to copy and paste text.
- Extending the system to perform real-time analysis of job posting URLs, including verification of company domain age and reputation.
- Extending the scope of the system to include resume fraud detection, identifying fraudulent or manipulated resumes submitted by applicants.
- Deploying the application on a cloud platform to make it accessible to a wider audience beyond a local or campus environment.

- Addressing class imbalance more directly through resampling techniques such as SMOTE, or through cost-sensitive learning, and comparing the resulting performance against the current approach.

Python Libraries Used

Library	Purpose in This Project
pandas	Provides the DataFrame structure used throughout the project for loading, cleaning, filtering, and transforming the tabular job posting dataset.
numpy	Supplies efficient array and numerical operations that underpin many Pandas and Scikit-learn computations, including handling of the TF-IDF feature matrices.
matplotlib	Used to generate static charts such as histograms and bar charts during Exploratory Data Analysis.
seaborn	Built on top of matplotlib, used for more visually refined statistical plots such as heatmaps, count plots, and boxplots during EDA.
scikit-learn (sklearn)	The core Machine Learning library used for TF-IDF vectorization, Label Encoding, Standard Scaling, train-test splitting, model training (Logistic Regression, Random Forest, SVM), GridSearchCV, and evaluation metrics.
nltk	Provides Natural Language Processing utilities used in the text preprocessing pipeline, including tokenization, stopword lists, and lemmatization.
re	Python's built-in regular expression module, used to remove punctuation, numbers, and other unwanted characters from text during cleaning.
joblib	Used to serialize (save) and deserialize (load) the trained model and TF-IDF vectorizer, allowing them to be reused instantly within the deployed Streamlit application without retraining.
streamlit	The web application framework used to build the interactive, browser-based user interface for the deployed prediction system.
reportlab	Used to programmatically generate the downloadable PDF prediction reports produced by the application.

Table 7: Python libraries used and their purpose

Machine Learning Concepts Explained

B.1 Train-Test Split

Train-test split refers to dividing the available labelled data into two separate subsets: one used to train the model (the training set) and one used only to evaluate the model after training (the testing set). This separation is essential to obtain an honest estimate of how the model will perform on new, unseen data, rather than an optimistic estimate based on data the model has already memorized.

B.2 Feature Scaling

Feature scaling refers to transforming numerical features so that they share a comparable scale, typically using techniques such as Standard Scaling (zero mean, unit variance) or Min-Max Scaling (values rescaled to a fixed range such as 0 to 1). Without scaling, features with naturally larger numeric ranges can disproportionately influence distance-based or margin-based algorithms such as SVM.

B.3 Label Encoding

Label Encoding is a technique that converts categorical text labels into numeric codes, assigning each unique category an integer value. This is necessary because most Machine Learning algorithms require numeric input and cannot directly process raw text categories.

B.4 TF-IDF (Term Frequency-Inverse Document Frequency)

TF-IDF is a numerical statistic used to represent text data for Machine Learning. It combines two components: Term Frequency, which measures how often a word appears within a specific document, and Inverse Document Frequency, which measures how rare or common that word is across the entire collection of documents (the corpus). Words that appear frequently in a specific document but rarely elsewhere receive a high TF-IDF score, making them useful discriminative features, while very common words that appear in almost every document (such as stopwords) receive a low score.

B.5 Classification

Classification is a supervised Machine Learning task in which the goal is to assign a predefined category or label to an input based on its features, using a model trained on previously labelled examples. In this project, classification is binary, assigning each job posting to either the “Genuine” or “Fraudulent” category.

B.6 Overfitting

Overfitting occurs when a model learns the training data too closely, including its noise and random fluctuations, resulting in excellent performance on the training set but poor generalization to new, unseen

data. Overfitting is a particular risk for complex models such as deep decision trees when trained on limited or noisy data.

B.7 Underfitting

Underfitting occurs when a model is too simple to capture the underlying patterns in the data, resulting in poor performance on both the training set and the testing set. It typically indicates that the chosen model or feature set lacks sufficient capacity or information to represent the true relationship between inputs and outputs.

B.8 Bias-Variance Tradeoff

The bias-variance tradeoff describes the balance between two sources of prediction error. High bias refers to error introduced by overly simplistic assumptions in the model (leading to underfitting), while high variance refers to error introduced by excessive sensitivity to small fluctuations in the training data (leading to overfitting). Effective model selection and tuning, such as the GridSearchCV process used in this project, aims to find a balance that minimizes total error on unseen data.

B.9 Hyperparameter Tuning

Hyperparameter tuning is the process of systematically searching for the combination of model configuration values, set before training begins, that yields the best model performance. Unlike model parameters (such as the weights learned during training), hyperparameters must be chosen by the practitioner or through an automated search procedure such as GridSearchCV, as described in Section 5.10.

B.10 Cross Validation

Cross validation is a technique for more reliably estimating a model's performance by splitting the training data into multiple folds, training the model on some folds and validating it on the remaining fold, and repeating this process so that every fold serves as the validation set exactly once. The results are then averaged to produce a more robust performance estimate than a single train-validation split, and this technique underpins the GridSearchCV process used for hyperparameter tuning in this project.

B.11 Feature Importance

Feature importance refers to a quantitative measure of how much each input feature contributes to a model's predictions. In tree-based models such as Random Forest, feature importance can be computed directly from how much each feature reduces impurity across all trees in the ensemble. Understanding feature importance, as discussed in Section 5.7.10, helps validate that a model is learning meaningful, domain-relevant patterns rather than arbitrary correlations.

References

[1] Kaggle, “Fake Job Posting Prediction Dataset,” Kaggle Datasets.

[2] Pedregosa, F. et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[3] McKinney, W., “Data Structures for Statistical Computing in Python,” Proceedings of the 9th Python in Science Conference, 2010.

[4] Bird, S., Klein, E., and Loper, E., Natural Language Processing with Python, O'Reilly Media, 2009.

[5] Cortes, C. and Vapnik, V., “Support-Vector Networks,” Machine Learning, vol. 20, pp. 273-297, 1995.

[6] Breiman, L., “Random Forests,” Machine Learning, vol. 45, pp. 5-32, 2001.

[7] Streamlit Documentation.